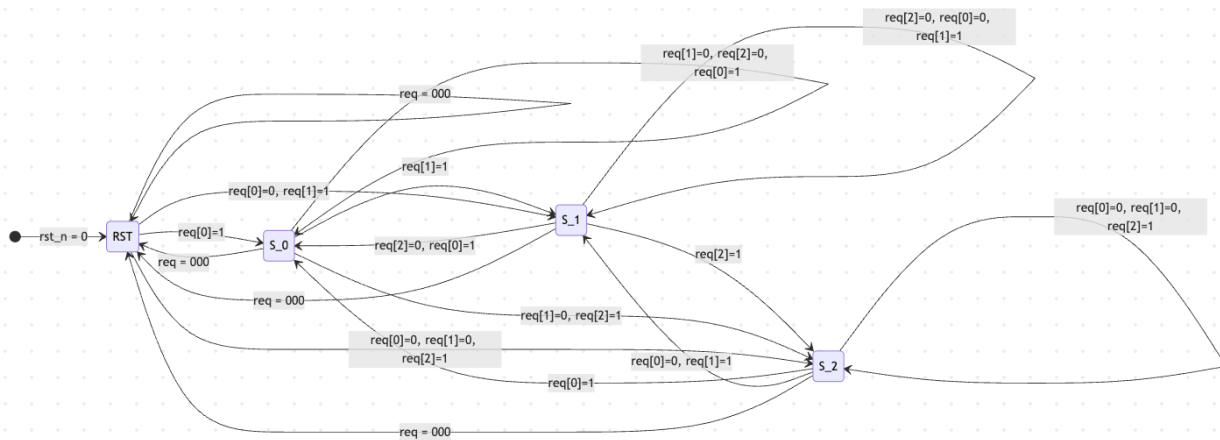


Lab 3 - Arbiter and FSM Design

PART 1 - Arbiter

1. Round-Robin Arbiter

State Machine Diagram



stateDiagram-v2

direction LR

[*] --> RST : rst_n = 0

%% 1. Primary Path (Highest Priority / Next in Round-Robin)

RST --> S_0 : req[0]=1

S_0 --> S_1 : req[1]=1

S_1 --> S_2 : req[2]=1

S_2 --> S_0 : req[0]=1

%% 2. Secondary Path (Skip one state)

RST --> S_1 : req[0]=0, req[1]=1

S_0 --> S_2 : req[1]=0, req[2]=1

S_1 --> S_0 : req[2]=0, req[0]=1

S_2 --> S_1 : req[0]=0, req[1]=1

%% 3. Tertiary Path (Skip two states)

RST --> S_2 : req[0]=0, req[1]=0, req[2]=1

```

%% 4. Self Loops (Hold current state)
RST --> RST : req = 000
S_0 --> S_0 : req[1]=0, req[2]=0, req[0]=1
S_1 --> S_1 : req[2]=0, req[0]=0, req[1]=1
S_2 --> S_2 : req[0]=0, req[1]=0, req[2]=1

%% 5. Return to Idle
S_0 --> RST : req = 000
S_1 --> RST : req = 000
S_2 --> RST : req = 000

```

Verilog Code (arb.v)

```

module arb (
    input wire      clk,
    input wire      rst_n, // active low and asynchronous
    input wire [2:0] req,
    output reg  [2:0] gnt
);

    parameter RST = 3'b000;
    parameter S_0 = 3'b001;
    parameter S_1 = 3'b010;
    parameter S_2 = 3'b100;

    reg [2:0] state, next_state;

    // State transition
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= RST;
        end else begin
            state <= next_state;
        end
    end

    // Next state logic
    always @(*) begin
        case (state)
            RST: begin
                if (req[0])      next_state = S_0;
                else if (req[1]) next_state = S_1;
                else if (req[2]) next_state = S_2;
                else              next_state = RST;
            end
            S_0: begin
                if (req[1])      next_state = S_1;
                else if (req[2]) next_state = S_2;
                else if (req[0]) next_state = S_0;
                else              next_state = RST;
            end
            S_1: begin
                if (req[2])      next_state = S_2;
            end
        endcase
    end

```

```

        else if (req[0]) next_state = S_0;
        else if (req[1]) next_state = S_1;
        else             next_state = RST;
    end
    S_2: begin
        if (req[0])      next_state = S_0;
        else if (req[1]) next_state = S_1;
        else if (req[2]) next_state = S_2;
        else             next_state = RST;
    end
    default: next_state = RST;
endcase
end

// Output logic
always @(*) begin
    gnt = state;
end

endmodule

```

Testbench Code (arb_tb.v)

```

`timescale 1ns/1ps

module arb_tb;

    reg clk;
    reg rst_n;
    reg [2:0] req;
    wire [2:0] gnt;

    // Instantiate the arbiter
    arb uut (
        .clk(clk),
        .rst_n(rst_n),
        .req(req),
        .gnt(gnt)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Function or string for state name
    reg [23:0] state_name;
    always @(*) begin
        case (uut.state)
            3'b000: state_name = "RST";
            3'b001: state_name = "S_0";
            3'b010: state_name = "S_1";
            3'b100: state_name = "S_2";
            default: state_name = "INV";
        end
    end
endmodule

```

```

        endcase
    end

    // Stimulus
    initial begin
        $dumpfile("arb.vcd");
        $dumpvars(0, arb_tb);

        // Initialize inputs
        rst_n = 0;
        req = 3'b000;

        // Wait for a few clock cycles before releasing reset
        #12;
        rst_n = 1;

        // Apply specified combinations of requests
        @(negedge clk) req = 3'b000;
        @(negedge clk) req = 3'b001;
        @(negedge clk) req = 3'b010;
        @(negedge clk) req = 3'b100;
        @(negedge clk) req = 3'b111;
        @(negedge clk) req = 3'b110;
        @(negedge clk) req = 3'b100;
        @(negedge clk) req = 3'b000;
        @(negedge clk) req = 3'b111;

        // Let it run for a few more cycles to observe behavior
        @(negedge clk);
        @(negedge clk);
        @(negedge clk);

        $finish;
    end

    // Clear printouts of grants at each clock cycle
    always @(posedge clk) begin
        // delay by a small amount to allow state and gnt to update
        #1;
        $display("Time=%0t | req=%b | gnt=%b | state=%s", $time, req, gnt,
state_name);
    end

endmodule

```

Simulation Output

```

Time=6000 | req=000 | gnt=000 | state=RST
Time=16000 | req=000 | gnt=000 | state=RST
Time=26000 | req=000 | gnt=000 | state=RST
Time=36000 | req=001 | gnt=001 | state=S_0
Time=46000 | req=010 | gnt=010 | state=S_1

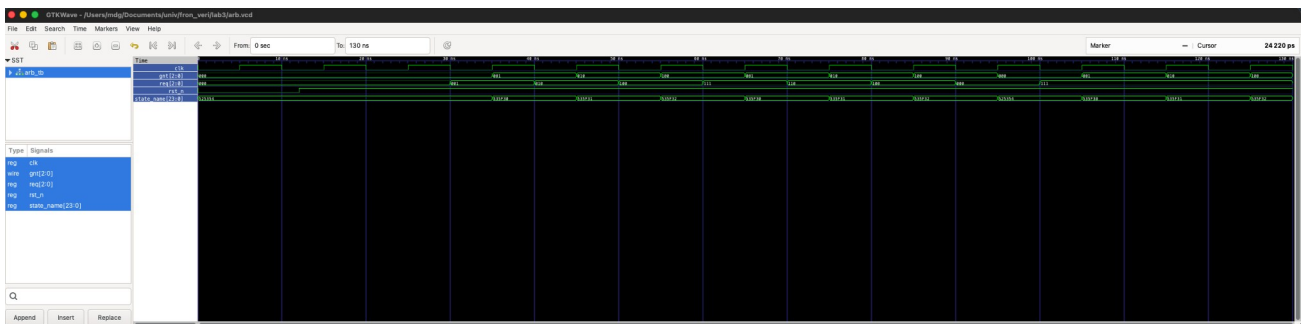
```

```

Time=56000 | req=100 | gnt=100 | state=S_2
Time=66000 | req=111 | gnt=001 | state=S_0
Time=76000 | req=110 | gnt=010 | state=S_1
Time=86000 | req=100 | gnt=100 | state=S_2
Time=96000 | req=000 | gnt=000 | state=RST
Time=106000 | req=111 | gnt=001 | state=S_0
Time=116000 | req=111 | gnt=010 | state=S_1
Time=126000 | req=111 | gnt=100 | state=S_2
arb_tb.v:65: $finish called at 130000 (1ps)

```

Waveform Images



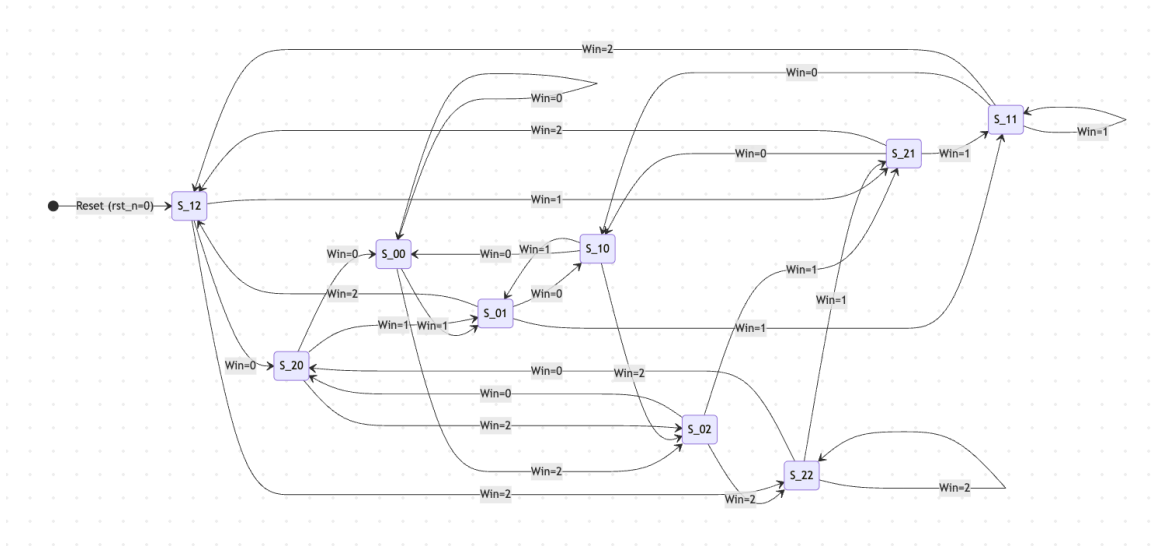
2. Smart(er) Arbiter

Design Explanation

The Smart Arbiter design does not use a traditional FSM with a single state variable. Instead, it utilizes two registers, `last_one_gnt` and `last_two_gnt`, to keep track of the history. To ensure absolute fairness, a combinational logic block assigns a 'priority score' to each requester based on the history:

- Score 2 (Highest): The requester is NOT present in the history registers.
- Score 1 (Medium): The requester was granted two cycles ago (`last_two_gnt`).
- Score 0 (Lowest): The requester was granted in the immediately preceding cycle (`last_one_gnt`).

When multiple requests arrive, the arbiter compares the active requesters' scores and grants access to the one with the highest score. In the event of a score tie, the system defaults to granting the requester with the lower ID number, ensuring a deterministic and fair decision.



stateDiagram-v2

direction LR

[*] --> S_12 : Reset (rst_n=0)

note "For any state, if req=000,\nit stays in the same state." as N1

```
%% -----
%% Group 0: Current Winner is 0 (States ending in '0')
%% Next state will always be S_0X
%% -----
```

S_00 --> S_00 : Win=0

S_10 --> S_00 : Win=0

S_20 --> S_00 : Win=0

S_00 --> S_01 : Win=1

S_10 --> S_01 : Win=1

S_20 --> S_01 : Win=1

S_00 --> S_02 : Win=2

S_10 --> S_02 : Win=2

S_20 --> S_02 : Win=2

```
%% -----
%% Group 1: Current Winner is 1 (States ending in '1')
%% Next state will always be S_1X
%% -----
```

S_01 --> S_10 : Win=0

S_11 --> S_10 : Win=0

S_21 --> S_10 : Win=0

```

S_01 --> S_11 : Win=1
S_11 --> S_11 : Win=1
S_21 --> S_11 : Win=1

```

```

S_01 --> S_12 : Win=2
S_11 --> S_12 : Win=2
S_21 --> S_12 : Win=2

```

```

%% -----
%% Group 2: Current Winner is 2 (States ending in '2')
%% Next state will always be S_2X
%% -----

```

```

S_02 --> S_20 : Win=0
S_12 --> S_20 : Win=0
S_22 --> S_20 : Win=0

```

```

S_02 --> S_21 : Win=1
S_12 --> S_21 : Win=1
S_22 --> S_21 : Win=1

```

```

S_02 --> S_22 : Win=2
S_12 --> S_22 : Win=2
S_22 --> S_22 : Win=2

```

Verilog Code (Smart_arb.v)

```

module Smart_arb (
    input wire      clk,
    input wire      rst_n,
    input wire [2:0] req,
    output reg [2:0] gnt
);

    reg [1:0] last_one_gnt;
    reg [1:0] last_two_gnt;

    reg [1:0] score0, score1, score2;
    reg [1:0] winner;

    // Calculate scores based on history
    // 2 = not in history (highest priority)
    // 1 = in last_two_gnt but not last_one_gnt
    // 0 = in last_one_gnt (lowest priority)
    always @(*) begin
        score0 = (last_one_gnt == 2'd0) ? 2'd0 : ((last_two_gnt == 2'd0) ? 2'd1
: 2'd2);
        score1 = (last_one_gnt == 2'd1) ? 2'd0 : ((last_two_gnt == 2'd1) ? 2'd1
: 2'd2);
        score2 = (last_one_gnt == 2'd2) ? 2'd0 : ((last_two_gnt == 2'd2) ? 2'd1
: 2'd2);
    end

```

```

// Determine the winner among active requesters
always @(*) begin
    case (req)
        3'b000: winner = 2'd3; // No request
        3'b001: winner = 2'd0; // Only req[0]
        3'b010: winner = 2'd1; // Only req[1]
        3'b100: winner = 2'd2; // Only req[2]
        3'b011: begin // req[0] and req[1]
            if (score0 >= score1) winner = 2'd0;
            else winner = 2'd1;
        end
        3'b101: begin // req[0] and req[2]
            if (score0 >= score2) winner = 2'd0;
            else winner = 2'd2;
        end
        3'b110: begin // req[1] and req[2]
            if (score1 >= score2) winner = 2'd1;
            else winner = 2'd2;
        end
        3'b111: begin // All requests
            if (score0 >= score1 && score0 >= score2) winner = 2'd0;
            else if (score1 > score0 && score1 >= score2) winner = 2'd1;
            else winner = 2'd2;
        end
        default: winner = 2'd3;
    endcase
end

// Update state and output
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // At reset, prefer lower number requester if no conflicts
        // So we set history such that 0 > 1 > 2
        last_one_gnt <= 2'd2;
        last_two_gnt <= 2'd1;
        gnt <= 3'b000;
    end else begin
        if (winner != 2'd3) begin
            last_two_gnt <= last_one_gnt;
            last_one_gnt <= winner;

            if (winner == 2'd0) gnt <= 3'b001;
            else if (winner == 2'd1) gnt <= 3'b010;
            else if (winner == 2'd2) gnt <= 3'b100;
        end else begin
            gnt <= 3'b000;
            // last_one_gnt and last_two_gnt remain unchanged when no
requests are given
        end
    end
end

endmodule

```


Testbench Code (smart_arb_tb.v)

```
`timescale 1ns/1ps

module smart_arb_tb;

    reg clk;
    reg rst_n;
    reg [2:0] req;
    wire [2:0] gnt;

    // Instantiate the smart arbiter
    Smart_arb uut (
        .clk(clk),
        .rst_n(rst_n),
        .req(req),
        .gnt(gnt)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Stimulus
    initial begin
        $dumpfile("smart_arb.vcd");
        $dumpvars(0, smart_arb_tb);

        // Initialize inputs
        rst_n = 0;
        req = 3'b000;

        // Wait for reset
        #12;
        rst_n = 1;

        // Apply scenarios to test history and fairness

        // Test 1: req 0 gets grant twice
        @(negedge clk) req = 3'b001;
        @(negedge clk) req = 3'b001;

        // Test 2: all requests asserted. Since 0 just got it twice, it should
        prefer lower number between 1 and 2, which is 1
        @(negedge clk) req = 3'b111;

        // Test 3: all requests asserted again. Now history is (0, 1), so 2
        should win
        @(negedge clk) req = 3'b111;

        // Test 4: all requests asserted again. History is (1, 2), so 0 should
        win
        @(negedge clk) req = 3'b111;
```

```

        // Test 5: req 1 and 2. History is (2, 0). 1 is not in history, so 1
should win
        @(negedge clk) req = 3'b110;

        // Test 6: No requests. gnt should be 000 and history unchanged.
        @(negedge clk) req = 3'b000;

        // Test 7: req 0 and 2. History is still (0, 1) from before req=000?
        // Wait, after req=110, history became (0, 1).
        // Let's see what happens here!
        @(negedge clk) req = 3'b101;

        // Additional cycles to observe the output
        @(negedge clk);
        @(negedge clk);

        $finish;
end

// Clear printouts of grants and history at each clock cycle
always @(posedge clk) begin
    // delay by a small amount to allow state and gnt to update
    #1;
    $display("Time=%0t | req=%b | gnt=%b | last_one=%0d | last_two=%0d",
            $time, req, gnt, uut.last_one_gnt, uut.last_two_gnt);
end

endmodule

```

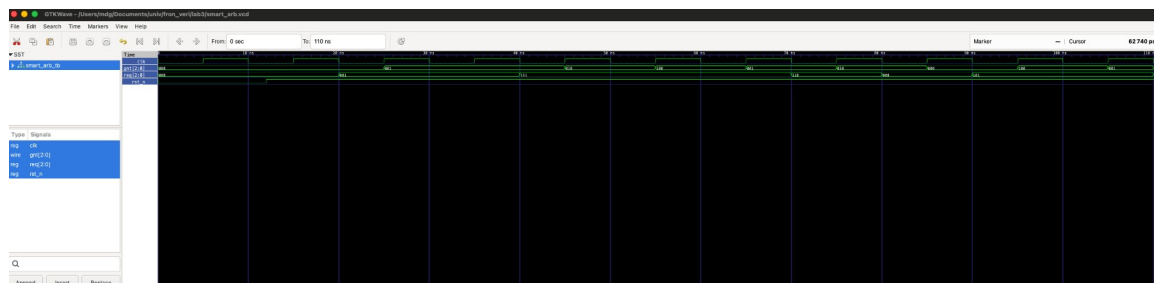
Simulation Output

```

Time=6000 | req=000 | gnt=000 | last_one=2 | last_two=1
Time=16000 | req=000 | gnt=000 | last_one=2 | last_two=1
Time=26000 | req=001 | gnt=001 | last_one=0 | last_two=2
Time=36000 | req=001 | gnt=001 | last_one=0 | last_two=0
Time=46000 | req=111 | gnt=010 | last_one=1 | last_two=0
Time=56000 | req=111 | gnt=100 | last_one=2 | last_two=1
Time=66000 | req=111 | gnt=001 | last_one=0 | last_two=2
Time=76000 | req=110 | gnt=010 | last_one=1 | last_two=0
Time=86000 | req=000 | gnt=000 | last_one=1 | last_two=0
Time=96000 | req=101 | gnt=100 | last_one=2 | last_two=1
Time=106000 | req=101 | gnt=001 | last_one=0 | last_two=2
smart_arb_tb.v:67: $finish called at 110000 (1ps)

```

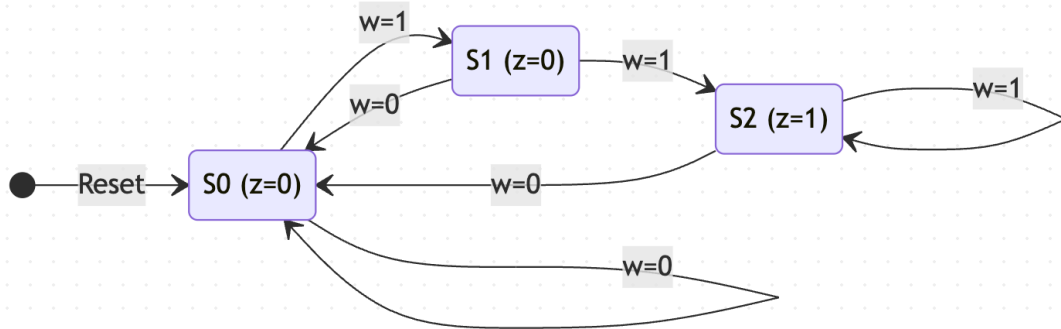
Waveform Images



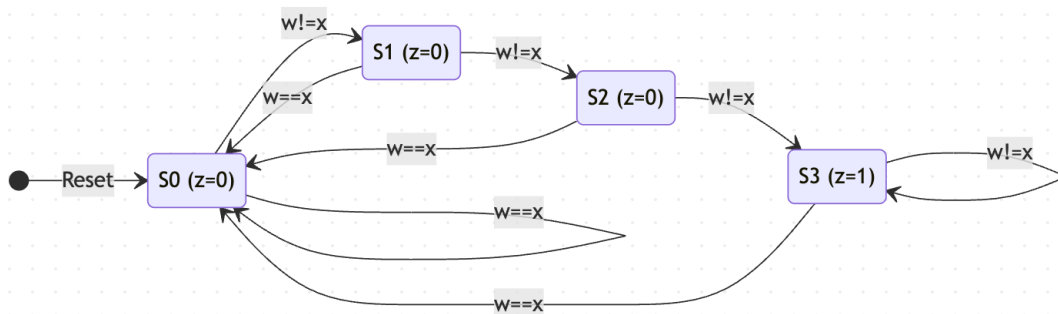
PART 2 - Datapath, Control, and Finite State Machines

State Machine Diagrams

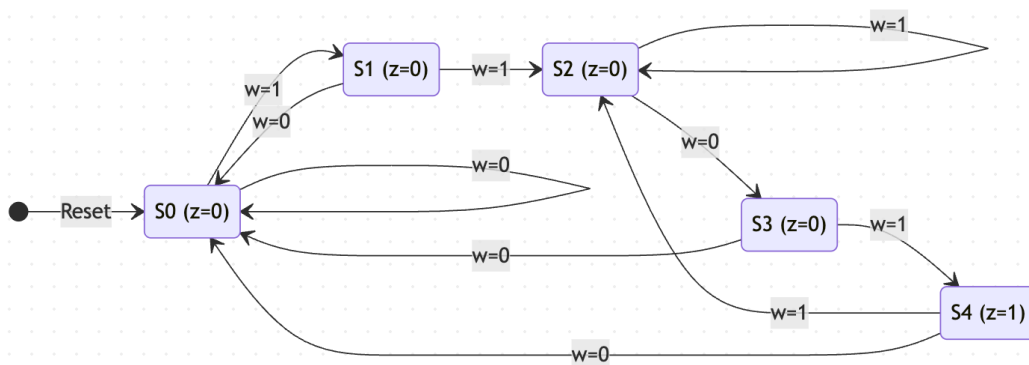
FSM for $w=1$ for two preceding cycles (Question 5)



FSM for $w \neq x$ for 3 consecutive cycles (Question 6)



FSM for sequence '1101' detection (Question 7)



Verilog Code (part2.v)

Contains edge detectors and all three FSMs.

```
module rising_edge_det (
    input wire clk,
    input wire button,
    output wire pulse
);
    reg button_d, button_dd;
    always @(posedge clk) begin
        button_d <= button;
        button_dd <= button_d;
    end
    assign pulse = button_d & ~button_dd;
endmodule

module falling_edge_det (
    input wire clk,
    input wire button,
    output wire pulse
);
    reg button_d, button_dd;
    always @(posedge clk) begin
        button_d <= button;
        button_dd <= button_d;
    end
    assign pulse = ~button_d & button_dd;
endmodule

module fsm_w (
    input wire clk,
    input wire rst_n,
    input wire w,
    output reg z
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
    reg [1:0] state, next_state;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= S0;
        else state <= next_state;
    end
    always @(*) begin
        case (state)
            S0: next_state = w ? S1 : S0;
            S1: next_state = w ? S2 : S0;
            S2: next_state = w ? S2 : S0;
            default: next_state = S0;
        endcase
    end
    always @(*) z = (state == S2);
endmodule
```

```

module fsm_wx (
    input wire clk,
    input wire rst_n,
    input wire w,
    input wire x,
    output reg z
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
    reg [1:0] state, next_state;
    wire diff = (w != x);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= S0;
        else state <= next_state;
    end
    always @(*) begin
        case (state)
            S0: next_state = diff ? S1 : S0;
            S1: next_state = diff ? S2 : S0;
            S2: next_state = diff ? S3 : S0;
            S3: next_state = diff ? S3 : S0;
            default: next_state = S0;
        endcase
    end
    always @(*) z = (state == S3);
endmodule

module fsm_l101 (
    input wire clk,
    input wire rst_n,
    input wire w,
    output reg z
);
    parameter S0 = 3'd0, S1 = 3'd1, S2 = 3'd2, S3 = 3'd3, S4 = 3'd4;
    reg [2:0] state, next_state;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= S0;
        else state <= next_state;
    end
    always @(*) begin
        case (state)
            S0: next_state = w ? S1 : S0;
            S1: next_state = w ? S2 : S0;
            S2: next_state = w ? S2 : S3;
            S3: next_state = w ? S4 : S0;
            S4: next_state = w ? S2 : S0;
            default: next_state = S0;
        endcase
    end
    always @(*) z = (state == S4);
endmodule

```

Testbench Code (part2_tb.v)

```
`timescale 1ns/1ps

module part2_tb;

    reg clk;
    reg rst_n;
    reg button;
    reg w_5, w_6, x_6, w_7;

    wire pulse_rise;
    wire pulse_fall;
    wire z_5;
    wire z_6;
    wire z_7;

    rising_edge_det u_rise (.clk(clk), .button(button), .pulse(pulse_rise));
    falling_edge_det u_fall (.clk(clk), .button(button), .pulse(pulse_fall));
    fsm_w u_fsm_w (.clk(clk), .rst_n(rst_n), .w(w_5), .z(z_5));
    fsm_wx u_fsm_wx (.clk(clk), .rst_n(rst_n), .w(w_6), .x(x_6), .z(z_6));
    fsm_1101 u_fsm_1101 (.clk(clk), .rst_n(rst_n), .w(w_7), .z(z_7));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("part2.vcd");
        $dumpvars(0, part2_tb);

        rst_n = 0;
        button = 0;
        w_5 = 0; w_6 = 0; x_6 = 0; w_7 = 0;
        #12;
        rst_n = 1;

        $display("--- Testing Edge Detectors ---");
        @(negedge clk) button = 1;
        @(negedge clk) button = 1;
        @(negedge clk) button = 0;
        @(negedge clk) button = 0;

        $display("--- Testing FSM 5 (w=1 for two preceding cycles) ---");
        // w: 0 1 0 1 1 0 1 1 1 0 1
        @(negedge clk) w_5 = 0;
        @(negedge clk) w_5 = 1;
        @(negedge clk) w_5 = 0;
        @(negedge clk) w_5 = 1;
        @(negedge clk) w_5 = 1;
        @(negedge clk) w_5 = 0;
        @(negedge clk) w_5 = 1;
        @(negedge clk) w_5 = 1;
        @(negedge clk) w_5 = 1;
    end
endmodule
```

```

    @(negedge clk) w_5 = 0;
    @(negedge clk) w_5 = 1;

    $display("--- Testing FSM 6 (w!=x for three cycles) ---");
    @(negedge clk) w_6 = 0; x_6 = 1; // 1 cycle
    @(negedge clk) w_6 = 1; x_6 = 0; // 2 cycles
    @(negedge clk) w_6 = 0; x_6 = 1; // 3 cycles -> z=1
    @(negedge clk) w_6 = 1; x_6 = 1; // 0 cycles
    @(negedge clk) w_6 = 0; x_6 = 1; // 1 cycle

    $display("--- Testing FSM 7 (1101) ---");
    @(negedge clk) w_7 = 1;
    @(negedge clk) w_7 = 1;
    @(negedge clk) w_7 = 0;
    @(negedge clk) w_7 = 1; // Sequence 1101 complete -> z=1
    @(negedge clk) w_7 = 1; // Sequence is now 11011 (suffix 11)
    @(negedge clk) w_7 = 0; // Suffix 110
    @(negedge clk) w_7 = 1; // Sequence 1101 again -> z=1

    @(negedge clk);
    @(negedge clk);
    $finish;
end

always @(posedge clk) begin
    #1;
    $display("Time=%0t | btn=%b rise=%b fall=%b | w5=%b z5=%b | w6=%b x6=%b
z6=%b | w7=%b z7=%b",
        $time, button, pulse_rise, pulse_fall, w_5, z_5, w_6, x_6, z_6,
w_7, z_7);
end

endmodule

```

Simulation Output

```

Time=6000 | btn=0 rise=0 fall=x | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
--- Testing Edge Detectors ---
Time=16000 | btn=0 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=26000 | btn=1 rise=1 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=36000 | btn=1 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=46000 | btn=0 rise=0 fall=1 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
--- Testing FSM 5 (w=1 for two preceding cycles) ---
Time=56000 | btn=0 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=66000 | btn=0 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=76000 | btn=0 rise=0 fall=0 | w5=1 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=86000 | btn=0 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=96000 | btn=0 rise=0 fall=0 | w5=1 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=106000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=116000 | btn=0 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=126000 | btn=0 rise=0 fall=0 | w5=1 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=136000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=146000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=156000 | btn=0 rise=0 fall=0 | w5=0 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0

```

```

--- Testing FSM 6 (w!=x for three cycles) ---
Time=166000 | btn=0 rise=0 fall=0 | w5=1 z5=0 | w6=0 x6=0 z6=0 | w7=0 z7=0
Time=176000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=0 | w7=0 z7=0
Time=186000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=1 x6=0 z6=0 | w7=0 z7=0
Time=196000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=0 z7=0
Time=206000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=1 x6=1 z6=0 | w7=0 z7=0
--- Testing FSM 7 (1101) ---
Time=216000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=0 | w7=0 z7=0
Time=226000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=0 | w7=1 z7=0
Time=236000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=1 z7=0
Time=246000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=0 z7=0
Time=256000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=1 z7=1
Time=266000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=1 z7=0
Time=276000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=0 z7=0
Time=286000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=1 z7=1
Time=296000 | btn=0 rise=0 fall=0 | w5=1 z5=1 | w6=0 x6=1 z6=1 | w7=1 z7=0
part2_tb.v:75: $finish called at 300000 (1ps)

```

Waveform Images

